DOINGG@UNION.EDU

COURSE: CSC106 CAN COMPUTERS THINK?

SELF

Contents

Home

The course website for CSC106, part of the Union College CS curriculum.
Here you can find our weekly schedule, assignmnets and resources.

Announcements

Project 1 due this Friday (4/17/26)

- 10% of total course grade
- will need to use Python documentation
- will be easier after doing texbook reading/exercises

Practical Exam next Thursday (4/23/26)

- 1 hr
- live coding **and** some written questions
- 10% of total course grade
- be sure to be caught up on texbook reading/exercises

Office Hours

- Student/Office Hours:
  - Steinmetz 108B
  - Monday 3:00 pm - 4:00 pm
  - Thursday 2:00 pm - 4:00 pm
  - subject to change, check course website for most up-to-date schedule
  - drop-in or schedule a 15 minute slot: https://calendar.app.google/8bus6pfDvyphR9ar5
  - by appointment for another time or over zoom

Course Description

Can computers think? Is it possible to build intelligent machines? What does it mean for a machine to think or to be intelligent? Why, or why not, would we want intelligent machines? To what extent do we already have them? How do they work? How do they impact our lives? What are their limitations? This course invites you to explore these questions and more.

This course is an introduction to the field of computer science with an artificial intelligence theme that does not assume you have prior experience with computer programming or computer science. It introduces basic algorithms, data structures and programming techniques. You will learn how computer scientists think about and solve problems by doing your own computer programming. This will give you an understanding of how the current technology for building intelligent machines works. In addition, you will read non-technical papers about artificial intelligence and the ethical questions raised by its use. You will reflect on the ways current computational systems often reinforce biases and can harm groups of people inequitably. And you will evaluate how the choices you are making when creating your own programs will affect future users.

If you're thinking about a CS or CPE major or minor, this course will give you a solid foundation to continue to *Programming on Purpose* (CSC 120),

to *Data Structures* (CSC 151) and beyond. If you're a neuroscience major, it will help you see how scientists are using biological and neurological principles to model "behavior'' in computers and it will teach you skills useful for processing experimental data. If you are thinking about another major or minor, this course will give you foundations to apply computing in whatever area you are interested in. And if you're here just because you're curious, that's great!

Schedule

Weekly Schedule

Schedule

check often as things may change

W	TOPIC	Due	Runestone
1	Programming, Calling Functions	Introductory Survey	2.1 - 2.13
2	Loops, Variables and Assignment	HW 1	3.1-3.5
3	Defining Functions	HW 2, Project 1	5.1-5.12
4	Conditional Statements	HW 3, Practical Exam 1	4.1-4.9
5	Conditional Loops	HW 4, Project 2	6.1-6.7
6	Strings, Lists	HW 5, Written Exam 1	7.1-7.12
7	Nested Lists	HW 6, Project 3	9.1-9.14
8	File I/O, Dictionaries	HW 7, Practical Exam 2	8.1-8.9
9	Dictionaries	HW 8, Project 4	10.1-10.6
10	Searching, Sorting, Recursion	HW 9, Written Exam 2	

Resources

CS-Ticket

- technical help with lab machines
- https://csticket.union.edu/

Runestone

- https://runestone.academy/runestone/default/user/register
- your Union College email
- unioncollege_py4e-int_spring26 as the course name

Gradescope

- https://www.gradescope.com/courses/1288185
- Entry Code: ZJ8ZRW

Late Token Form

- https://forms.gle/WXW44b2AM6kQCdX16
- fill out this form to get a 24 hour extension on any Programming Project
- each student gets 3 tokens per term

Python

- Python 3.13.X: the software package needed to write and run Python programs
- https://www.anaconda.com/download/success?reg=skipped

Python Documentation

- Official Documentaion: https://docs.python.org/3/index.html
- Can *always* be used as a reference
- Can be searched for module-specific documentation e.g. turtle

Reeborg

- Reeborg's World: a useful website for learning some programming basics
- Reeborg's World: https://reeborg.ca/reeborg.html
- Reeborg Project Documentation

PythonTutor

- Python Tutor: a useful website for tracing python code
- https://pythontutor.com/visualize.html#mode=edit

Google Colab Pro

- Pro account sign-up: https://colab.research.google.com/signup
- Use your Union email
- A pro account is not needed for this course, just an option Union pays for
- Notes template: https://colab.research.google.com/drive/15rPmKyQKCCbwsdDCpC39o8hczWZ-B5zV?usp=sharing

Python Style Guide

- PEP 8
- should be used in all code cells of Colab notebooks
- Link: https://peps.python.org/pep-0008/

Syllabus

Syllabus

- check for updates! (All updated text in this posted version of the syllabus will supersede text from older versions.)

CSC106 Syllabus

1 CSC 106: Can Computers Think?

- Union College, Spring 2026
- Georgia Doing, PhD: email: doingg@union.edu, office: Steinmetz 108B

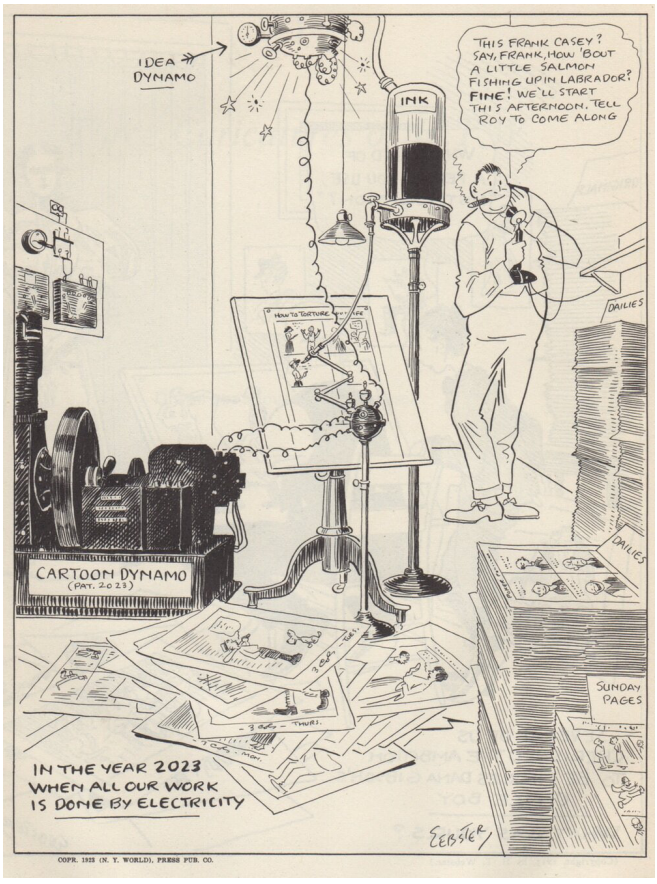


Figure 1: Cartoon by H.T. Webster published in 1922.

2 COURSE BASICS

- Lecture: Monday, Wednesday, Friday 1:50 pm - 2:55 pm
- Lab: Thursday 10:55 am - 12:40 pm
- Location: Olin 107
- Student/Office Hours:
  - Monday 3:00 pm - 4:00 pm
  - Thursday 2:00 pm - 4:00 pm
  - subject to change, check course website for most up-to-date schedule
  - drop-in or schedule a 15 minute slot: https://calendar.app.google/gnYNtHjwwNxJ2UUN7
  - by appointment for another time or over zoom

3 OVERVIEW

Can computers think? Is it possible to build intelligent machines? What does it mean for a machine to think or to be intelligent? Why, or why not, would we want intelligent machines? To what extent do we already have them? How do they work? How do they impact our lives? What are their limitations? This course invites you to explore these questions and more.

This course is an introduction to the field of computer science with an artificial intelligence theme that does not assume you have prior experience with computer programming or computer science. It introduces basic algorithms, data structures and programming techniques. You will learn how computer scientists think about and solve problems by doing your own computer programming. This will give you an understanding of how the current technology for building intelligent machines works. In addition, you will read non-technical papers about artificial intelligence and the ethical questions raised by its use. You will reflect on the ways current computational systems often reinforce biases and can harm groups of people inequitably. And you will evaluate how the choices you are making when creating your own programs will affect future users.

If you're thinking about a CS or CPE major or minor, this course will give you a solid foundation to continue to *Programming on Purpose* (CSC 120), to *Data Structures* (CSC 151) and beyond. If you're a neuroscience major, it will help you see how scientists are using biological and neurological principles to model "behavior' ' in computers and it will teach you skills useful for processing experimental data. If you are thinking about another major or minor, this course will give you foundations to apply computing in whatever area you are interested in. And if you're here just because you're curious, that's great!

4 LEARNING OBJECTIVES

By the end of the course, you should be able to *answer* the following questions:

- What are the most important programming fundamentals and why are they the building blocks of programs?
- What are the debugging fundamentals and how do you use them?
- What are some of the mechanisms used to build "intelligent" machines?

By the end of the course, you should be able to *do* the following:

- Use variables, functions, conditionals, loops and recursion
- Read Python programs
- Write short Python programs that solve a given problem
- Fix short Python programs using debugging fundamentals

Computing/programming topics we will cover include functions, variables, assignment, conditionals, loops, recursion, lists/arrays, file reading/writing, binary numbers and ASCII, strings and string methods and dictionaries.

A 1-page schedule-at-a-glance is included at the end of the syllabus, but please see Nexus for a link to a more detailed schedule.

5 CLASS POLICIES AND GUIDELINES

The Computer Science Department as a whole welcomes all people, regardless of age, background, beliefs, ethnicity, gender identity, gender expression, national origin, religious affiliation, sexual orientation and any other differences, be they visible or non-visible. It's also important to recognize that institutional racism has prevented members of marginalized groups - especially black, indigenous and other people of color (BIPOC) - from fully participating in the field of Computer Science.

You. Yes you, the one reading this syllabus. You belong here.

As an instructor, I will do my utmost to uphold these principles and to treat everyone with respect. As students in this class, I expect you to do the same since one person is not a community unto themselves. We're all in this together, so let's treat each other well. Lastly, I welcome feedback on issues we might discuss or ways that our class can be a more just one for all people.

6 COURSE MATERIALS

Online Textbook: Python for Everybody - Interactive (free).

We are going to use an online, interactive, free and open source textbook. This textbook has a lot of embedded interactive exercises. Reading this book always also means doing the activities. To get access to the book, please register on Runestone using the following informationm which can also be found on Nexus:

- https://runestone.academy/runestone/default/user/register
- your Union College email
- unioncollege_py4e-int_spring26 as the course name

We'll use the following websites and software throughout the course:

- The course website: a github-hosted site with assignments, materials and announcements
  - https://georgiadoing.github.io/CSC106-S26/
  - you may want to bookmark this to check it before and after each class
- Gradescope: used to submit programming assignments
  - https://www.gradescope.com/courses/1288185
  - Entry Code: ZJ8ZRW
- Python 3.13.X: the software package needed to write and run Python programs
- Reeborg's World: a useful website for learning some programming basics
- https://reeborg.ca/index_en.html
- Python Tutor: a useful website for tracing python code
  - https://pythontutor.com/visualize.html#mode=edit

7 COURSE RESOURCES

The course website has a list and links to the readings and software resources that we are going to use. In class, we are going to use the Linux machines in Olin 107. Outside of class you have a choice or hardware. You can install the software on your own computer (Python is freely available for Windows, Mac, and Linux, Reeborg's World and Python Tutor are browser based programs) or you can work in one of the Computer Science labs. We have three spaces that you can use 24/7 using your ID card, except when classes are being held in them:

- Olin 107 - where we have class
- PASTA Lab (ISEC 051+)
- Computer Science Resource room (Steinmetz Hall, 209A)

8 LIBRARY RESOURCES

The library is available to help you with your research needs! Librarians can help you develop research questions, search for and select the best sources for your projects, identify research strategies, evaluate sources, and assist you with creating citations. There are multiple ways for you to contact a librarian. For more information, please see our Ask A Librarian page: https://www.union.edu/schaffer-library/ask-a-librarian.

9 ACCOMODATIONS

It is the policy of Union College to make reasonable accommodations for qualified individuals with disabilities. If you are a person with a disability and wish to request accommodations to complete your course requirements, please make an appointment with me or stop by during my office hours as soon as possible, all discussions will remain confidential. You must provide reasonable notice and be in touch with Accomodative Services on the 2nd floor of Schaffer Library, if you have not already, if you wish to take advantage of extra time on exams.

10 COMPLEX QUESTIONS: GLOBAL CHALLENGES & SOCIAL JUSTICE

Justice, Equity, Identity, Difference (JEID). Algorithms and computational systems increasingly shape the world we live in. Advances in AI and machine learning are in the news on a daily basis. But there is also a growing awareness of the limitations and dangers of these systems. In particular, they often promote biases and inequalities. For example, facial recognition software has been shown to work much less accurately for people of color than for white people, an AI hiring algorithm was found to reject female applicants for technical positions at a higher rate than male applicants, and search results can be full of racial stereotypes. In this course, you will learn about real-world cases of AI systems that discriminate against groups of people. You will see that related ethical questions are raised even in the context of the relatively simple programs you create in an introductory computer science class. Additionally, you will work on developing a habit of always evaluating how your own choices as a programmer might impact other people's experiences.

Engineering, Technology and Society (ETS). In this course, you learn to break down a larger problem into a series of smaller computational steps (creating an algorithm) and to implement algorithms in Python. Through these activities you will gain an understanding of how computers work and an idea of the software development process. In particular, you will learn about iterative development and the importance of thorough testing. You will practice communicating about algorithms and their implementation in groups.

You will also explore, through assignments and discussion, how computational systems can have unintended and negative consequences for groups of people and what the responsibility of the programmer is to mitigate these dangers.

Data & Quantitative Reasoning (DQR). Much of current AI systems are data driven and learn based on collections of data. In this course, you will learn about and implement some approaches that AI systems use to draw inferences based on data. Many of the ethical issues around current AI systems are linked to the data these systems use. Throughout the course, you will, therefore, also reflect on: What information is collected? How is the information represented? Who is represented in the data? How is it used?

11 GRADING

The following components will contribute to your final grade for this class with roughly the indicated weight.

- 2 practical exams: 20% (10% each)
- Written midterm: 15%
- Written final exam: 15%
- Programming projects: 40%
- Engagement: 10%

Note: You must get a C- or better in order to take any other class that has a CSC-10x prerequisite. Several larger programming projects will be assigned to reinforce the concepts discussed in class and put into practice in homework assignments. All projects can be completed using programming techniques that have been covered in class.

Do not use techniques that have not been covered in class unless specifically told otherwise. Part of the challenge for each programming project is figuring out how to complete the assignment using only the tools provided in the course up to that point.

Letter Grade Scale:

- A: 93-100; A-: 90-92
- B+: 87-89; B: 83-86; B-: 80-82
- C+: 77-79; C: 73-76; C-: 70-72
- D: 60-69; F: 0-59

12 ATTENDANCE

Class participation is a critical component of the course and attendance is mandatory. I realize that sometimes other things come up (interviews, athletic events, illness, emergencies etc.).

In those cases, just let me know (in advance, if at all possible) that you are going to be absent. There may not always be the possibility of make-up credit for missed in-class material, but it is more likely for something to be arranged if you discuss your absence with me prior to class. No matter how much class you have missed, please do not come to class if you have an illness that is likley contageous, please email me to work out a reasonable solution.

If you do miss class, it is your responsibility to make up any material that you missed. Get notes from a classmate, make sure to complete all homework assignments due or assigned during the class that you missed, and come see me if you have any questions on the material. I will expect you to hand in all assignments by the due date, regardless of whether you were in class. You will not be able to make up missed exams or in-class engagement points without pre-approved exceptions.

13 LATE POLICIES

Homework assignments will usually be discussed in class on the day that they are due and will not be accepted late. Programming projects are due by 11:59:59pm on the due date. You have three "extension tokens" for this class. Each token will extend a single project deadline by 24 hours. You can use each token on a different project, or use two or even all three on a single project. Once all three tokens have been used, no late submissions will be accepted (barring exceptional circumstances).

14 PARTICIPATION AND ENGAGEMENT

Following these guidelines will constitute postiive participation and engagement and earn you **up to 1% of your final grade per week**. To demonstrate comprehension of in-class and/or homework assignments and earn relevent engagement points, students will submit written answers for collection and adhere to the follwoing guidelines. **Falling short of any of the below standards may result in point deductions from any engagement activities relevent to class time(s) during which the student was unengaged**.

- Respect. This course is a space for rigorous and respectful debate. When confronting ideas and people different from you, lead with curiosity rather than judgement. We will critique ideas, not people. To protect our shared learning environment, you may not record, photograph, or share any part of our class sessions outside of our class.
- Engage. Show up to class on time and prepared. Show up ready to learn by completing the readings and exercises and checking the course website often. Engagement in class constitutes being present, respectful and lending the class your full attention. **It is the student's responsibility to demonstrate this engagement by completing and handing-in thoughtful written responses as directed by in-class prompts and/or**

homework assignments. Accessing off-topic websites or software, checking email or phones, *utilizing large language models* or otherwise attending to things not pertinant to the course is distracting to yourself and others and will result in point deductions.

- Embrace intellectual humility. Recognize that there are limitations to your knowledge and that some of your beliefs could actually be wrong. Be curious about your thinking and open to learning from others. This is hard, but so vital to your learning in this course—we'll work on developing this throughout the term!
- Get help when you need it. If you are stuck or confused or lost, be proactive and get help. The best learners and thinkers can figure out when they are stuck and make a plan to get unstuck. Come to Office Hours (Steinmetz 108B), the CS Helpdesk (Sun-Thurs 7:00-9:00pm Olin 107), Biology Backup (ISEC 316, Tues & Thurs 5:30-7:30pm) or ask another student. Often the best person to explain something is the person who just figured it out. Try reaching out to another student in the course who seems like they get it—they will probably be flattered you asked!

15 ACADEMIC INTEGRITY AND "ARTIFICAL INTELLIGENCE" USE

Union College recognizes the need to create an environment of mutual trust as part of its educational mission. Responsible participation in an academic community requires respect for and acknowledgement of the thoughts and work of others, whether expressed in the present or in some distant time and place. Matriculation at the College is taken to signify implicit agreement with the Academic Honor Code, available at: http://muse.union.edu/honorcode

It is each student's responsibility to ensure that submitted work is their own and does not involve any form of academic misconduct. Students are expected to ask their course instructors for clarification regarding, but not limited to, collaboration, citations, and plagiarism. Ignorance is not an excuse for breaching academic integrity. Students are also required to affix the full Honor Code Affirmation, or the following shortened version, on each item of coursework submitted for grading:

'I affirm that I have carried out my academic endeavors with full academic honesty.'

[Signed, Jane /John /Jaden Doe]

What it means for this course: **you must complete the programming projects on your own, but you are welcome and encouraged to collaborate on other regular assignments.**

Please notice that different rules apply to the exercises that are part of the textbook, regular assignments and programming projects.

Textbook Readings and Regular Exercises:

You are welcome to discuss the textbook readings, the exercises embedded in the textbook, and other regular assignments with other people in the class to help you fully understand the

material. However, don't just copy somebody else's solution. Even if you ask other people for help, the goal is for you to understand the underlying concepts and logic. It's more important to understand how to arrive at a correct solution than it is to know the solution itself.

Programming Projects:

Any work you turn in has to be your own. The best way to learn programming is by doing programming. I expect that you already know that if you do not do things for yourself, you will not learn them. Sometimes that will mean that you will have to struggle. That is ok: struggling to figure out how to solve a problem is where a lot of learning happens. Give yourself the opportunity to do this.

Please, do not ever copy any part of a solution from a friend, from any material generated by an "artificial intellence" (AI) chatbot model (ChatGPT, Gemini, Copilot, Claude, etc.) or from anywhere else. Do not ever *look at* somebody else's solution or any solution generated by an entity other than yourself before you have completed your own. Do not ever *show* somebody else your solution before you've both submitted it; though it may be beneficial to discuss your solution with another student in the class after you've both handed in the assignment. Please do not *ever* show your solution to future students of this class or any chatbot, and do not *ever* post your solutions on the Internet.

Don't search for information on the Internet. Don't ask for information from any AI model. Especially, don't look at any material that provides solutions to exercises that are similar to the projects. An exception is, of course, any online material that is linked directly from the Nexus page. As always, you should practice good citation behavior and cite any sources that you consult in your work (for example, which pages from the Python documentation website you consulted when working on the project).

Discussing your work (without looking at code): Talking through a problem is extremely beneficial in understanding the problem and coming up with a solution path. So, I strongly encourage you to talk to other students in this class about the projects, visit the CS Helpdesk and come to my Office Hours (you can also request a meeting if you have schedule conflicts with my Office Hours).

Here are some ways in which you can talk about the projects without having to worry about violating the rules from above.

- It's ok (and strongly encouraged!) to draw **memory diagrams** together. Draw memory diagrams and discuss possible algorithms for solving the problem.
- It's ok to **write down an algorithm in English** together. Then go off by yourself to implement it into code.
- It's ok to read and write code together that is not part of the assignment, but that helps demonstrate the concept of what you're being asked to do. Go through an example from class or from the book together, or **come up with your own examples**. There's no better way to understand something than trying to think up examples in order to teach it to someone else. Try it!

Do not work on the implementation together with friends, especially if you are each at your own computer and are always in lock step trying to figure out the same line or section of code at the same time. Have a higher-level discussion (using one of the strategies suggested above), then work on the implementation on your own.

Your goal for discussions about any assignment should be that you come away with a better understanding of the problem and of possible ways to approach it so that you can then try out these approaches on your own. You should never leave a discussion with just an answer, without an understanding of how to arrive at that answer. A good general guideline for any discussion, or interaction with an AI model, is that you should not leave the discussion with anything written or typed.

16 CONTACTING ME OUTSIDE OF CLASS

The best method for contacting me outside of class is to stop by my office during Office Hours or whenever my office door is open. You can also schedule a time with me if my regular office hours don't work. Please contact me via email (doingg@union.edu) and we can arrange a phone or zoom call. I respond to emails as soon as possible, but it may take up to one day to get a response during the term, and longer between terms.

17 ADDITIONAL RESOURCES

Mental Health and Campus Resources

Union College is committed to supporting and advancing the mental health and well-being of our students. During the course of their academic careers, students often experience personal challenges that contribute to barriers in learning, such as drug/alcohol problems, strained relationships, chronic worrying, persistent sadness or loss of interest in enjoyable activities, family conflict, grief and loss, domestic violence, difficulty concentrating, problems with organization, procrastination and/or lack of motivation. Students also sometimes come to college with a history of learning difficulties (e.g., any form of special education), experience difficulties succeeding in a particular subject (e.g., math, reading), or have experienced some form of trauma be it emotional or physical (e.g., head injury). These mental health concerns can lead to diminished academic performance and can interfere with daily life activities. If you or someone you know has a history of mental health concerns or if you are unsure and would like a consultation, a variety of confidential services are available. The Eppler-Wolff Counseling Center provides free counseling and therapy services (including psychiatry) to all enrolled students. Please call (518) 388-6161 or visit the Wicker Wellness Center in person any weekday between 8:30 and 5:00 to schedule an initial contact appointment. Visit the Counseling Center website for more information.

In a crisis situation, or after hours, contact Campus Safety at (518) 388-6911. The National Suicide Prevention hotline also offers a 24-hour hotline at (800) 273-8255.

Any student who faces challenges securing their food or housing and believes this may affect their performance in the course is urged to contact the Dean of Students (dos_office@union.edu) for support or drop into office hours Monday - Friday 8:30 am - 5:00 pm in the Reamer Campus Center room 306. Furthermore, please notify me if you are confortable doing so and I will provide any resources I can. The Union College Persistance Fund can provide financial support in times of unexpected harship to cover needs such as emergency medical expenses not covered by insurance and basic living expenses. Additional sources of supoprt are outlines on the Dean of Students webpage: https://www.union.edu/dean-students .

18 TENTATIVE SCHEDULE

W	TOPIC	Due	Runestone
1	Programming, Calling Functions	Introductory Survey	2.1 - 2.13
2	Loops, Variables and Assignment	HW 1	3.1-3.5
3	Defining Functions	HW 2, Project 1	5.1-5.12
4	Conditional Statements	HW 3, Practical Exam 1	4.1-4.9
5	Conditional Loops	HW 4, Project 2	6.1-6.7
6	Strings, Lists	HW 5, Written Exam 1	7.1-7.12
7	Nested Lists	HW 6, Project 3	9.1-9.14
8	File I/O, Dictionaries	HW 7, Practical Exam 2	8.1-8.9
9	Dictionaries	HW 8, Project 4	10.1-10.6
10	Searching, Sorting, Recursion	HW 9, Written Exam 2	

18.1 ADDENDA

1. Updated text in the section on "Participation and Engagement" section explaining what types of evidence will be collected and assessed to measure engagement in accordance with the grading scheme. (4/17/26)
2. Updated text in "Attendance" to allow for in-class engagement points to be made-up with pre-approved exceptions. Exams cannot be made-up but it may be possible to take them early with pre-approved exceptions. (4/17/26)
3. All updated text in this syllabus will supersede text from older versions. (4/17/26)

19 PROGRESS TRACKER

Learning Goals	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10
What are the most important programming fundamentals and why are they the building blocks of programs?										
What are the debugging fundamentals and how do you use them?										
What are some of the mechanisms used to build “intelligent” machines?										
Use variables, functions, conditionals, loops and recursion.										
Write short Python programs that solve a given problem.										
Fix short Python programs using debugging fundamentals.										

Grading

Table 2: Grading Scheme

Course Element	Grade Point Contribution	Notes
Engagement	10%	written answers collected and scanned, 0.25% per activity: itemized list
Programming Projects	40%	4 at 10% each
Midterm	15%	written
Practical Exams	20%	coding & written
Final Exam	15%	written
total	100%	

Table 3: Letter Grade Scale

Letter Grade	Percent range
A	93 - 100
A-	90 - 92
B+	87 - 89
B	83 - 86
B-	80 - 82
C+	77 - 79

Letter Grade	Percent range
C	73 – 76
C-	70 – 72
D	60 – 69
F	0 – 59

Note: You must get a C- or better in order to take any other class that has a CSC-10x prerequisite. Several larger programming projects will be assigned to reinforce the concepts discussed in class and put into practice in homework assignments. All projects can be completed using programming techniques that have been covered in class.

Do not use techniques that have not been covered in class unless specifically told otherwise. Part of the challenge for each programming project is figuring out how to complete the assignment using only the tools provided in the course up to that point.

Assignments

HW 01

CSC106: Assignment 1 (0.25 pts)

- due Monday 4/6/26 by class time (1:50 pm)
- in-class notes from Friday: https://drive.google.com/file/d/1UW3fMha6Ki4dDR_sidpeiV-PDeBSFEPi/view?usp=sharing
  - examples of comments, importing turtle, variables and for loops
  - also includes link to turtle library documentation

Part 1

- Intro Survey
- Runestone Chapters 1-3 (we'll discuss in class next week starting Monday)

Part 2

- Practice Problems for Week 1: HW01 instructions
- Starter code: winding_sandbar.json
- Submit code on Gradescope
- bring written answers to class (0.25 engagement points awarded on these)
- Graded on effort/completion

HW 02

CSC106: Assignment 2 (0.25 pts)

- due Monday 4/13/26 by class time (1:50 pm)
- Runestone Chapter 5 (see schedule for specific sub-sections)
- Practice Problems for Week 2:
  - Submit code on Gradescope,
  - Bring written answers to class (0.25 engagement points awarded on these)
  - Graded on effort/completion

NAME:

Practice Problems for Week 2

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Monday 4/13/26**.

You will be submitting the following files:

- square_chain.py
- retirement.py
- *All the programming files **MUST** be named as instructed*

Programming Part 1: Turtle Stairs

1. Create a file called square_chain.py for your code.
2. Write a Python program using turtle to draw something that draws a diagonal chain of 10 squares:

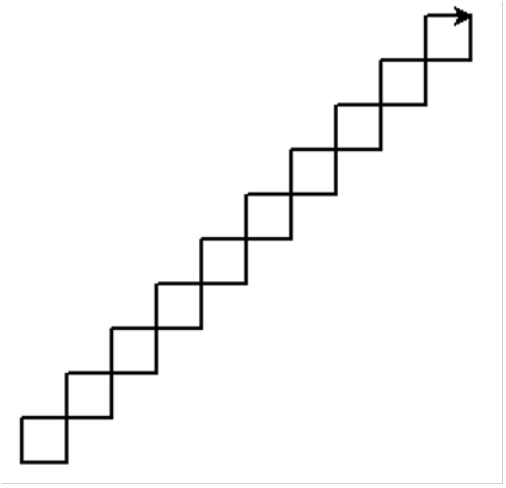


Figure 1: Turtle Stairs

There are many different ways (algorithms) that you can draw this pattern. Some may (or may not) leave the turtle in the same position and orientation that you see here. It does not matter if the turtle has the same position and orientation as what you see here.

3. Add comments to explain how your program works.
    - Header comment with Name, Email, Date, Assignment, Description
    - In-line comments describing the algorithm guiding your program
    - Docstrings for any functions, if you define new functions

Programming Part 2: Retirement Calculator

It's never too early to start saving for retirement. For this assignment you're going to write a simple retirement calculator.

- A run of the program may look like the following (user input in bold):

```
How much do you plan to save each month? (in dollars) **100**
How old are you? (in years) **20**
At what age do you plan to retire? (in years) **40**
What yearly rate of return do you expect? (in percentage points) **5**
Your savings at age 21 are: $ 1200
Your savings at age 22 are: $ 2460
Your savings at age 23 are: $ 3783
Your savings at age 24 are: $ 5172
Your savings at age 25 are: $ 6630
Your savings at age 26 are: $ 8162
Your savings at age 27 are: $ 9770
Your savings at age 28 are: $ 11458
Your savings at age 29 are: $ 13231
Your savings at age 30 are: $ 15093
Your savings at age 31 are: $ 17048
Your savings at age 32 are: $ 19100
Your savings at age 33 are: $ 21255
Your savings at age 34 are: $ 23518
Your savings at age 35 are: $ 25894
Your savings at age 36 are: $ 28388
Your savings at age 37 are: $ 31008
Your savings at age 38 are: $ 33758
Your savings at age 39 are: $ 36646
Your savings at age 40 are: $ 39679
```

1. Start by creating a file called retirement.py for your code.
2. Next, start writing code to get input from the user and store the information in variables.

- The program should start by asking the user for the following information:
  - The amount they plan to save each month
  - Their starting age
  - Their planned retirement age
  - The yearly rate of return they expect (i.e. growth through interest or dividends).

3. Now design an algorithm to compute the amount that accumulates each year. Write out the algorithm in a human language (e.g. English) before writing it in code. Include this as your description in your header comment.

- The program should show how the user's savings grow over the course of their careers. For example, if a 20 year-old starts saving $100 a month, retires at 40, and gets a 5% rate or return, then they should have about $39,679 by the time they retire.

- To simplify things, we assume that interest is added once per year before the user has contributed any of their monthly savings.
  - This means that, for the first year, the rate of return is zero, so they only get what they saved over the course of the year (i.e. $1200).
  - For subsequent years, calculate the interest gained, then add the yearly contribution. So for year 2:
    - the interest will be $1200 × 5% = $60
    - the savings at the end of year will be $1200 + $60 + $1200= $2460.

4. Implement the program in Python
5. Test the program with different values to ensure that it works correctly.
6. If you haven't already, add comments explaining how the program works.

Written Questions

1. Consider the following (incomplete) Python program:

```
x = int(input("Please input a positive integer: "))
counter = x
for step in range(0, 10):
```

Complete the missing lines of code above so that the program will produce the following output if the user inputs 3.

```
3
6
9
12
15
18
21
24
27
30
```

- **Notes**: You do not need to introduce any additional variables. There is more than one way to achieve this result.

2. Consider the following (incomplete) Python program to list the first few Fibonacci numbers:

The Fibonacci numbers are the following sequence of numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, …

- The first two numbers are 0 and 1.
- After that, each number can be calculated by adding the previous two together. For example:
  - the third number is 1 (0+1)
  - the fourth number is 2 (1+1)
  - the fifth number is 3 (2+1), etc.

They are named Fibonacci numbers after the Italian mathematician who first described them in Europe. Patterns based on the Fibonacci sequence show up in surprisingly many contexts in math, biology, and art. (See the Wikipedia entry if you want to know more.)

Below is the (incomplete) Python program to list the first few Fibonacci numbers.

The user gets to determine how many Fibonacci numbers to list.

```
1.  print ("How many Fibonacci numbers would you like to see?")
2.  limit = int (input("Please input a number greater than 2: "))
3.
4.  fib_0 = 0
5.  fib_1 = 1
6.
7.  print(fib_0)
8.  print(fib_1)
9.  for count in range(        ):
10.    next_fib =
11.    print(next_fib)
```

- For questions a-d, assume that the program has been completed and functions correctly.

2.a. Draw a memory diagram that shows the- associations after line 5 has been executed. Assume that the user inputted 10. Also show what will have been printed to the screen at that point.

2.b. Draw a memory diagram that shows the - associations that should exist after line 11 has been executed for the first time. Also show what line 11 will print to the screen when it is executed for the first time.

2.c. What should the - associations be when line 11 has been executed for the second time? Draw a memory diagram.

2.d. Draw a memory diagram that shows the - associations after the last number has been printed (i.e. at the end of the program).

2.e. Complete the code above so that the program prints the first n Fibonacci numbers, where n is determined by the user. You may need to add new lines of code.

3. Consider two Python programs:

- Both programs on the next page will help Reeborg harvest the carrots in the Harvest 1 world:

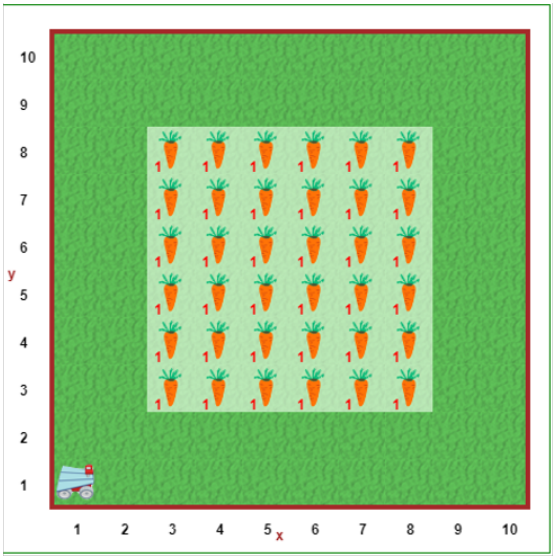


Figure 2: Reeborg Carrots

- Both programs define a number of functions. Unfortunately, the program designers were from the bad old days when names needed to be as short as possible, so their function names are essentially meaningless. Come up with better names for each of the functions that they defined.

a. .....
b. .....
c. .....
d. .....
e. .....
f. .....
g. .....
h. .....

```
# Program 1

1  def turn_right():
2   turn_left()
3   turn_left()
4   turn_left()
5
6  def a():
7   move()
8   turn_left()
9   move()
10  move()
11  turn_right()
12  move()
13
14 def b():
15  for carrot in range(0, 6):
16      take()
17      move()
18
19 def c():
20  turn_left()
21  move()
22  turn_left()
23  move()
24
25 def d():
26  turn_right()
27  move()
28  turn_right()
29  move()
30
31 a()
32 for double_row in range(0, 3):
33  b()
34  c()
35  b()
36  d()
```

```
# Program 2

1  def turn_right():
2   turn_left()
3   turn_left()
4   turn_left()
5
6  def e():
7   move()
8   turn_left()
9   move()
10  move()
11  turn_right()
12  move()
13  take()
14
15 def f():
16  for carrot in range(0, 5):
17      move()
18      take()
19
20 def g():
21  f()
22  turn_left()
23  move()
24  turn_left()
25  take()
26
27 def h():
28  f()
29  turn_right()
30  move()
31  turn_right()
32  take()
33
34 e()
35 for double_row in range(0, 2):
36  g()
37  h()
38 g()
39 f()
```

3.i Now that you've come up with better names for the functions, which of the solutions do you think is better and why?

……

……

……

……

……

……

……

……

……

……

……

……

4. Check-in

4.a. Were you able to complete all of the programming activities (yes/no)?

……

4.b. What, if anything, did you struggle with when working on the activities?

……

……

……

……

4.c. If you managed to overcome the challenges, how did you do it?

……

……

……

……

Project 1

CSC106: Project 1

- due Friday 4/17/26 by class time (1:50 pm)
- you may use late token(s) for this assignment

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

- Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.
- Projects are graded on program correctness and code composition, including proper commenting
- This project is 10% of your final grade
- starter code:
  - square.py
  - turtle_shapes.py
  - computer_art.py

NAME:

Project 1: Drawings with Turtle

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

Learning Objectives:

1. Practice designing and implementing simple algorithms.
2. Demonstrate that you can use the programming concepts that we've covered so far: function calls, function definitions, variables and assignment, and counting loops
3. Practice reading the official Python documentation.
4. More practice defining your own functions and using them.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Drawing shapes (4 pts)

1.1 Square

1. Open the starter code `square.py`. You'll notice that a few variables are defined at the top of the file. Please use these variables in your code.
2. Add some code that moves the turtle to the (`x`,`y`) position without drawing a line.
3. Add some code that draws the square using size as the side length.
4. Make sure to save your program in a file called `square.py`.
5. Make sure that your code is fully commented.

1.2 Rectangle

1. Create a new code file called rectangle.py using a similar format to `square.py`. Meaning:
    a. Define `x, y` to indicate the starting position for the rectangle.
    b. Create: width and height to control the size of the rectangle.
    c. The entire rectangle should be visible in the turtle window without having to resize it, but beyond this restriction, feel free to place the rectangle wherever you like and to make it as big or small as you like.
2. Add code to move the turtle to position (`x`,`y`).
3. Add code to draw a rectangle with the specified width and height.
4. Save this code in a file called `rectangle.py` and make sure to comment the code.
    - Hint: It may be very helpful to copy and paste code from `square.py` to reduce the amount of typing you have to do.

1.3 Circle

1. Follow the same general steps for the rectangle, except this time you'll be creating a circle.
    a. File name: `circle.py`.
    b. Variables to create: `x, y, radius`.

1.4 Triangle

1. Follow the same general steps as for the rectangle, but this time we're making a triangle.
    a. File name: `triangle.py`.
    b. Variables to create: `x, y, size`.
2. This program should draw an equilateral triangle.
3. The size variable should indicate the length of one side.

2, Functions for shapes (1 pt)

1. Open the starter file `turtle_shapes.py`. This file contains the beginnings of some function definitions. (We'll be discussing function definitions more this week)
2. Copy your code for drawing a square from `square.py` into the `square(x, y, size)` function definition. There are comments explaining how to do this in the starter file.
    - Caution: Do not copy the variable definitions at the top of the square.py file. They will not be needed if you designed your square.py code correctly.
3. You should notice that the `square(x, y, size)` function is called at the bottom of the starter file. Once you've completed the previous step, running the program should produce a drawing of a square in the bottom right quadrant of the turtle window.
4. Repeat step 2 for the other shapes (rectangle, circle, triangle).
    - Caution: Don't change the order of the parameters in the function definitions.
5. Add function calls for each of the shapes at the bottom of the file to test your program and prove that the functions work as expected.
6. Please note that you should copy over comments in addition to the code itself.

3. Draw a picture (2 pts)

1. Create a new file and copy the function definitions from Part 2 into it. Save this new file as `my_picture.py`.
2. Write a program that uses these functions to draw a picture.
3. Picture requirements:
    a. Use at least two line colors and at least two fill colors.
    - Hint: it may be useful to refer back to your in-class assignments. If you're still having trouble, take a look at the `turtle` library section of the Python official documentation (https://docs.python.org/3/library/turtle.html)… or stop by my office hours.
    b. Your picture should depict a recognizable object or scene. For example, something like the picture bloew (though it does not need to be this involved)

c. You may add additional functions to help draw the picture (e.g. functions to draw filled-in versions of each shape), but you should not change the functions that you defined for Part 2 of the assignment.
d. Your program should use each function from Part 2 at least once, but you can also use other built-in turtle functions.
e. Your program should use loops when appropriate to keep your code DRY (Don't Repeat Yourself).

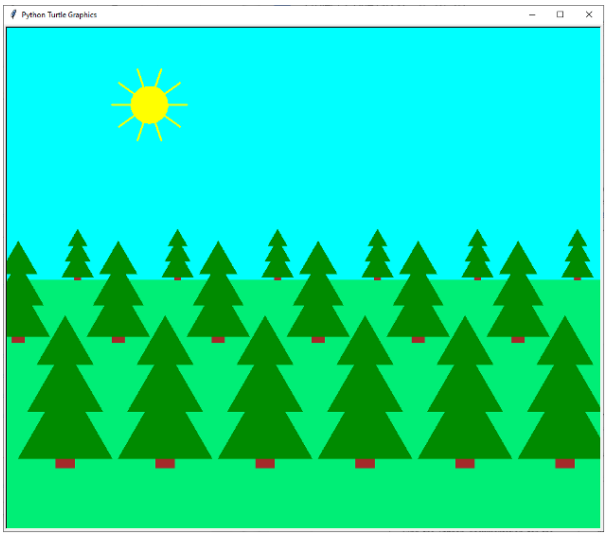


Figure 1: Turtle Drawing

4. Computer Generated art (3 pts)

1. Find the Python documentation for the `random` module. (Hint: Use the link to the "library reference" on the Resources page on the course website)
2. The function `randrange` from the `random` module produces a random integer. Test it out by writing a program that generates five random numbers between 2 and 9 (inclusive) and prints them out. Save your program in a file called `five_random_numbers.py`. Hint: remember to keep your code DRY.
3. Open the starter file `computer_art.py`.
4. This file already contains definitions for two functions. In particular, it contains a function called `random_shape` that randomly picks a shape (square, rectangle, circle, or triangle) and draws it. The function takes three parameters:

    a. `x` - the x location
    b. `y` - the y location
    c. `size` - the size

5. Copy and paste your function definitions from Part 2 into this file.
6. Add some code to draw 30 randomly picked shapes at random locations on the screen. For example, your output could look something like this:

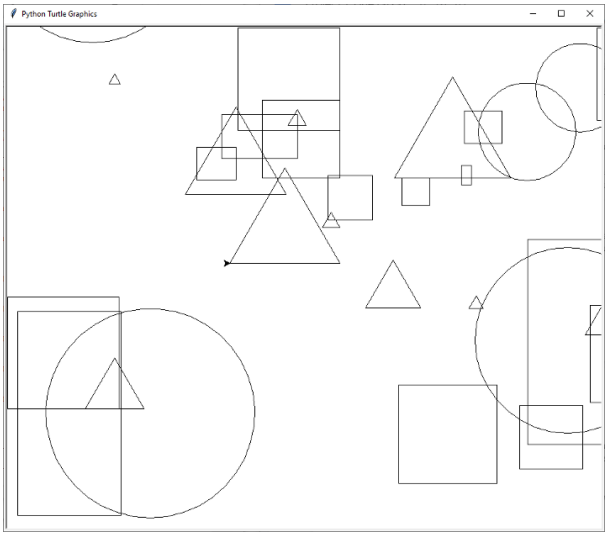


Figure 2: Computer Art

7. (Optional): Add some code to randomly change the pensize, pencolor, and filcolor.

5. What to submit

You will need to submit the following files:

1. `square.py`
2. `rectangle.py`
3. `circle.py`
4. `triangle.py`
5. `turtle_shapes.py`
6. `my_picture.py`
7. `five_random_numbers.py`
8. `computer_art.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:
    a. A header comment, which includes your name, the date, the assignment and a brief description of the program.
    b. Comments within the body of the code to further clarify how the program works.
2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code?
    - Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?
    a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting.

Once you have checked these things, submit your code to:

"cat3_Project 1: Drawings with Turtle" on Gradescope.

6. Grading guidelines

- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.

HW 03

CSC106: Assignment 3 (0.25 pts)

- due Monday 4/20/26 by class time (1:50 pm)
- Runestone Chapter 5 (see schedule for specific sub-sections)
- Practice Problems for Week 3:
  - Submit code on Gradescope,
  - Bring written answers to class (0.25 engagement points awarded on these)
  - Graded on effort/completion

starter code

- birthday.py
- criket.py

NAME:

Practice Problems for Week 3

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Monday 4/20/26**.

You will be submitting the following files:

- birthday.py
- cricket.py

Programming Part 1: Happy Birthday

1. Download the starter file `birthday.py` from the course website and read the instructions inside before continuing.
2. Write a function that "sings" Happy Birthday for someone. The function should take the person's name as a parameter and then print Happy Birthday in the following format.:

```
Happy Birthday to you!
Happy Birthday to you!
Happy Birthday dear <name>!
Happy Birthday to you!
```

3. Make sure to include appropriate comments.
4. Save your program using the filename `birthday.py`.

Programming Part 2: Cricket Chirping

1. Download the starter file `cricket.py` from the course website and read the instructions inside before continuing.
2. Amos Dolbaer discovered that, for certain types of crickets, there's a relationship between the temperature and the rate at which they chirp (https://en.wikipedia.org/wiki/Dolbear's_law). More specifically, the relationship is as follows:

$$T = 50 + (N - 40)/4$$

where T is the temperature in Farenheight and N the number of chirps per minute.

Write a function that calculates the temperature when given the number of times a cricket chirps in 15 seconds. Your function should do the following:

a. calculate the chirps per minute
b. calculate the temperature based on chirps per minute
c. return the temperature

3. Save your program in a file called `cricket.py`

Written Questions

1. 1. Which of the following are valid first lines for a Python function definition? Please explain why the invalid ones are not valid.

a. `def plus ten (anumber):`

b. `def double (anumber)`

c. `def greeting:`

d. `percentage(total, part):`

e. `def print_introduction():`

f. `def move()`

g. `def goto(x coordinate y coordinate):`

2. Consider the Python program below:

```
1)  def sum_to_n(n):
2)       total = 0
3)       for count in range(0, n+1):
4)            total += count
5)       return total
6)
7)  print(sum_to_n(6))
```

a. Draw a memory diagram that shows what memory looks like after the Python interpreter has executed line 2.

b. Draw a memory diagram that shows what memory looks like after the Python interpreter has executed line 4 for the first time.

c. How many times will line 4 be executed in total?

d. Draw a memory diagram that shows what memory looks like right before the Python interpreter executes line 5.

e. Draw a memory diagram that shows what memory looks like right after the Python interpreter executes line 7.

3. Print or return?

Consider the following (incomplete) function definitions.

Given the way these functions are used in the code at the end, complete the dotted lines by either turning them into a statement that **returns** the indicated value or into a statement that **prints** the indicated value.

Also complete the dotted lines in the code at the end by choosing to either `return` or `print` the value of the variable `a`.

```
def f1():
    a = int(input('Please input a number: '))

    ..................a........

def f2(var):
        squared = var**2

    ..................squared........

def f3(var):

    ..................var**3........

a = 10

..................a........
f2(a)
print(f1() + 3)
a = f3(a) * 5

..................a........
```

f. Assuming that the user input 1 when asked for a number, what will this code print to the screen?

.....

.....

.....

.....

.....

.....

4. Check-in

4.a. Were you able to complete all of the programming activities (yes/no)?

……

4.b. What, if anything, did you struggle with when working on the activities?

……

……

……

……

4.c. If you managed to overcome the challenges, how did you do it?

……

……

……

……

HW 04

CSC106: Assignment 4 (0.25 pts)

- due Friday 5/1/26 by class time (1:50 pm)
- Runestone Chapter 6 (see schedule for specific sub-sections)
- Practice Problems for Week 4:
  - Submit code on Gradescope,
  - Bring written answers to class (0.25 engagement points awarded on these)
  - Graded on effort/completion

starter code

- sum_of_powers.py

NAME:

Practice Problems for Week 4

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Wednesday 4/29/26**.

You will be submitting the following files:

- sum_of_powers.py
- primes.py

Programming Part 1: Sums of Powers

Sums of powers arise in many contexts in maths s. A sum of powers is a sequence of the first n integers raised to a given power. For example, the sum of the first 5 squares is: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$

1. Download the starter file sum_of_powers.py from the course website.
2. Write a function `called sum_of_powers` which, given an exponent and an integer n that indicates how many numbers to sum up, calculates (and returns) the first n integers raised to the given exponent.
3. Save your program in a file called `sum_of_powers.py`.
4. Select a few inputs for your function and calculate what the correct output of your function should be for those inputs by hand (or with a calculator). Use these inputs to test your function.

Programming Part 2: Primes

A prime number is a natural number greater than 1 that is only evenly divisible by itself and 1. In this assignment you will define a function called is_prime that checks whether a given number is a prime number.

1. Create a file called `primes.py`, which you'll use for this assignment.
2. Before writing any code, answer the following questions. Add the answers as comments to the top of your file.
    a. How many parameters does this function need and what do they represent?
    b. Should this function have a return value? If so, what kind of value will be returned? (e.g. number, string, boolean?)
    c. Give a few example function calls that you could use to test whether your implementation works correctly.
3. The name of the function you create should be is_prime. Below is an algorithm you can use for your function.Note: You only have to worry about positive integers.
    a. Assume that x is the number that you're checking to determine if it's prime.

   b. If x is smaller than 2, it is not a prime number, so you can return False immediately.
   c. Otherwise, define a boolean variable, which represents whether or not x is prime. Initialize this variable to (i.e. assign it the value) True. Note: in effect, we begin by assuming that x is prime.
   d. For all numbers between 2 and x, check whether x is evenly divisible by the number. If so, x is not prime. Record this by changing the value of the boolean variable you defined in the previous step.
   e. At the end, the value of the boolean variable should reflect whether or not x is prime, so all you need to do is return that value.

4. (Optional) Can you improve on this algorithm to make it more efficient? Hint 1: Aside from x itself, what is the largest number x can possibly be evenly divisible by? Hint 2: Are there any numbers between 2 and x that you don't need to check?

Written Questions

1. Which of the following are expressions? Which of the expressions are Boolean expressions (ie. conditions)? What value is described by each of the Boolean expressions? Assume that the following variables have been defined:

```
x = 7
y = 2
```

Consider the following problem specification: Define a function that doubles a numeric (int or float) parameter. Which of the following successfully implement this function and demonstrate to a user that it works? For those that don't satisfy both requirements, explain what the problem(s) is/are.

```
1.  def double(a):
        result = 2*a
        return result

    print(result)
```

.....
.....
.....
.....
.....

```
2.  def double(a):
        result = 2*a
        return result

    double(10)
    print(result)
```

.....
.....
.....
.....
.....

```
3.  def double(a):
        result = 2*a
        return result

    double(10)
    print(return)
```

.....
.....
.....
.....
.....

4.
```
def double(a):
    result = 2*a
    return result

x = double(10)
print(x)
```

5.
```
def double(a):
    result = 2*a
    return result

x = double(10)
print(double(10))
```

6.
```
def double(a):
    result = 2*a
    return result

print(double(10))
```

7.
```
def double(a):
    result = 2*a
    return result

x = double(10)
print(double(10))
```

8.
```
def double(a):
    result = 2*a
    print result

print(double(10))
```

9.
```
def double(a):
    result = 2*a
    print result

double(10)
```

Project 2

CSC106: Project 2

- due Monday 5/4/26 by 11:59 pm
- you may use late token(s) for this assignment

In this project, you'll implement several variants of a text-based Rock-Paper-Scissors game in which a human player plays against the computer. If you're not familiar with the game, check out the Wikipedia article about it: (http://en.wikipedia.org/wiki/Rock-paper-scissors)

- Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or course website for what to do when you need help with your work.
- Projects are graded on program correctness and code composition, including proper commenting
- This project is 10% of your final grade
- starter code:
  - rps_1.py
  - rps_2.py
  - game_history.py
  - rps_anti_human.py
  - rps_human_like.py

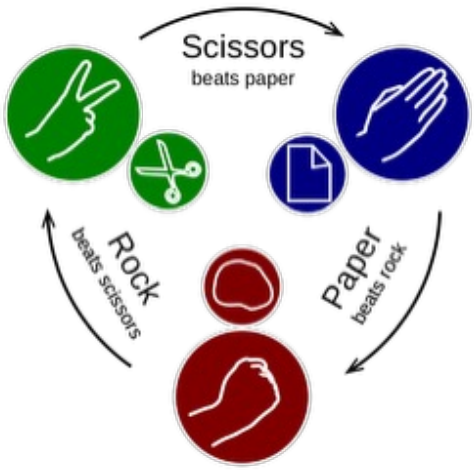
NAME:

Project 2: Rock Paper Scissors (10 pts)

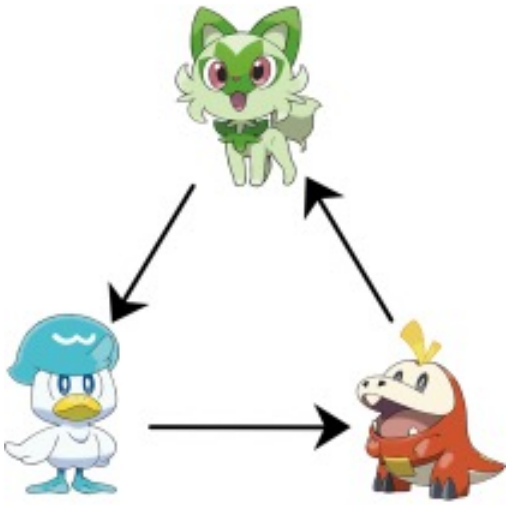
In this project, you'll implement several variants of a text-based Rock-Paper-Scissors game in which a human player plays against the computer. If you're not familiar with the game, check out the Wikipedia article about it: (http://en.wikipedia.org/wiki/Rock-paper-scissors)

Although Rock-Paper-Scissors is a simple game, mechanisms that function similarly to it have been studied in a variety of fields ranging from game theory to biology. One of the most common places to see rock-paper-scissors-like mechanics though is in video games. For example, many interactions between different "types" in the Pokémon games are similar to a game of rock-paper-scissors - the most classic of these being the interactions between the three Pokémon the player can choose at the very beginning of each game in the main series: the options being grass, fire, and water.

The version you'll implement here involves three choices - rock, paper, and scissors. Paper beats (wraps around) rock, scissors beat (cut) paper, and rock beats (blunts) scissors.



(a) RPS



(b) RPS

Learning Objectives:

1. Demonstrate that you can use the programming concepts that we've covered so far. In particular: function calls and definitions, variables and assignment, conditional statements.
2. Practice using random numbers to make random and random, but biased choices.
3. Explore ways to imitate human behavior with the computer.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Basic Rock-Paper-Scissors

1.1 Playing one game (2 pts)

1. Open the starter code `rps_1.py` from the course website.
2. Write a function called `rps` that plays a game of rock-paper-scissors with the user. The computer first randomly generates its own move (rock, paper, or scissors) and then asks the user for their choice. Next, it confirms what the user has chosen, reveals its own choice and finally makes one of the following announcements: `"It's a tie", "I win", or "You win"`
3. Make sure that your code is fully commented. Here are a few example interactions:

```
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1 [Enter] <<< This is the prompt for user input.
You chose: 1 (rock)
I chose: 1 (rock)
It's a tie.
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1
You chose: 1 (rock)
I chose: 2 (paper)
I win.

Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    3
You chose: 3 (scissors)
You chose: 2 (paper)
You win.
```

Hints:

- You probably will want to use the Python `random` module. You can use functions from the module that you've used before (i.e. randrange) or you could use other functions such as randint or choice.
- A link to the documentation for built-in Python modules can be found on the resources page on the course website.
- Feel free to define additional functions if you find it helpful to do so. However, please make sure that `rps` will produce one whole run through the game as shown in the example interactions above.

1.2 Playing until you're tired of it (2 pts)

1. Download the starter file `rps_2.py` from the course website. This file contains a definition for a function called `play`. When play is called with another function name as the parameter value, it will call this function repeatedly until the user indicates that they no longer want to continue.
2. Copy and paste the code for your `rps` function from `rps_1.py` into `rps_2.py`. Follow the instructions in the comments. When you run `rps_2.py`, you should see an interaction similar to the following:

```
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    2
You chose: 2 (paper)
You chose: 3 (scissors)
I win.
Do you want to play again? Type y or n.    y
Great!
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1
You chose: 1 (rock)
You chose: 3 (scissors)
You win.
Do you want to play again? Type y or n.    n
Ok. See you next time. Bye, bye!
```

2. RPS variations

Choose two (2) of the following four options. For each option, start by making a copy of `rps_2.py`, but save the new code under a new name, as specified by the option you're working on.

2.1 Cheating (3 pts)

Please use the following filename for this variant: `rps_cheating.py`.

In your original RPS program, the computer should choose its move before it asks the human player for their move. But, there's no way for the human player to check whether this is actually what's happening in the program or not. The computer could cheat by waiting for the human to choose a move, then pick a move that beats the human's move.

In this variant, that's what the computer should do. But to not be too obvious, the computer should not cheat every time. Each time the computer has to pick a move, there should be a 1/3 chance that it will cheat. Otherwise, it chooses the move randomly.

2.2 Exploiting human biases (3 pts)

For this variant, download the starter files `rps_anti_human.py` and `game_history.py`. Make sure to save both of these files in the same folder. You'll write all of your code in the file `rps_anti_human.py`.

Studies of humans playing rock-paper-scissors have shown that human players do not always pick their moves randomly. Instead, after they've won, they're more likely to repeat the same move in the next round. In contrast, when they lose, humans are more likely to pick a move to beat the move that they just lost to. For example, if they just played rock and lost to paper, they have a tendency to pick scissors in the next round in order to beat paper.

Knowing about this behavior pattern, you can devise a strategy to beat human players:

1. If I (the computer) just lost (and my human opponent won), I know that the human is likely to play the same move they played in the last round. So I will choose my next move such that it beats their last move.
2. If I (the computer) just won (and my human opponent lost), I know that the human is likely to shift their move to beat my last move. So I will pick my move accordingly. Because of the way rock-paper-scissors works, that means I should now play whatever the human played in the last round.

To implement this strategy, you need a way of remembering the outcome of the last round and what moves were made. The starter file contains a number of functions that you can use for that purpose. These functions refer to code from `game_history.py`. Please do not make any changes to `game_history.py` or to the given function definitions.

References: https://arstechnica.com/science/2014/05/win-at-rock-paper-scissors-by-knowing-thy-opponent/, https://arxiv.org/abs/1404.5199

2.3 Imitating human behavior (3 pts)

For this variant, download the starter files `rps_human_like.py` and `game_history.py`. Make sure to save both of these files in the same folder. You will write all of your code in `rps_human_like.py`.

Studies of humans playing rock-paper-scissors have shown that human players do not always pick their moves randomly. Instead, after they've won, they're more likely to repeat the same move in the next round. In contrast, when they lose, humans are more likely to pick a move to beat the move that they just lost to. For example, if they just played rock and lost to paper, they have a tendency to pick scissors in the next round in order to beat paper.

In this variant, the computer should imitate human behavior. That is:

- If the computer won the last round, it should have a tendency to repeat the same move. However, to not be too predictable, it should only do so with 50% probability. Otherwise it will pick its next move randomly.
- If the computer lost the last round, it should have a tendency to pick a move that defeats the last move by the human player. But again, to not be too predictable, it will only do so with a 50% probability. Otherwise, it will pick its next move randomly.
- If the last game ended in a tie, the computer picks the next move randomly. To implement this strategy, you need a way of remembering the outcome of the last round and what moves were made. The starter file contains a number of functions that you can use for that purpose. These functions refer to code from `game_history.py`. Please do not make any changes to `game_history.py` or to the given function definitions.

2.4 Five options: rock, paper, scissors, Spock, lizard (3 pts)

Start from `rps_2.py` and save it as `rps_five.py`. In this variant you will be implementing a variant of rock-paper-scissors called rock-paper-scissors-Spock-lizard. As the name suggests, it involves 5 options:

- rock beats (blunts) scissors and (crushes) lizard
- paper beats (covers) rock and (disproves) Spock
- scissors beat (cut) paper and (decapitate) lizard
- Spock beats (smashes) scissors and (vaporizes) rock
- lizard beats (poisons) Spock and (eats) paper

Make adjustments to the code originally written for `rps_2.py` so that it implements these five options instead of the original three.

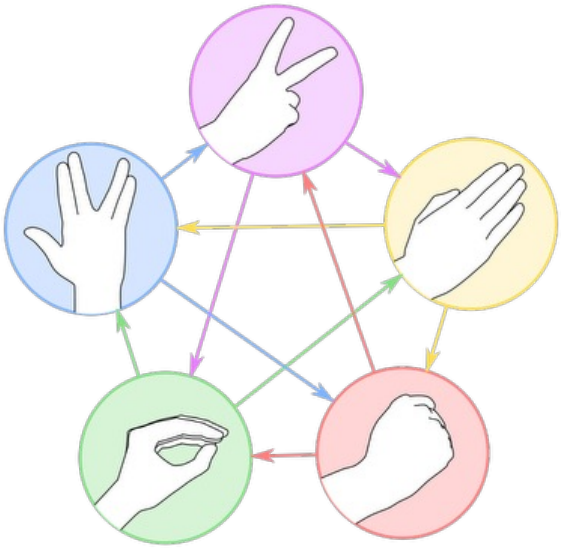


Figure 2: RPS Five

3. What to submit

What to submit You will need to submit the following files:

- `rps_1.py`
- `rps_2.py`

And two of the following (depending on which RPS variants you decide to implement):

- `rps_cheating.py`
- `rps_anti_human.py`
- `rps_human_like.py`
- `rps_five.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:

a. A header comment, which includes your name and a brief description of the program.
b. Comments within the body of the code to further clarify how the program works.

2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code? Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?

a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting. Once you have checked these things, submit your files to *cat3_Project 2: Rock-Paper-Scissors* on Gradescope.

6. Grading guidelines

- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.

HW 05

CSC106: Assignment 5 (0.25 pts)

- due Friday 5/5/26 by class time (1:50 pm)
- Runestone Chapter 7 (see schedule for specific sub-sections)
- Practice Problems for Week 5:
  - Submit code on Gradescope,
  - Bring written answers to class (0.25 engagement points awarded on these)
  - Graded on effort/completion

starter code

- number_guessing.py

NAME:

Practice Problems for Week 5

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Wednesday 5/5/26**.

You will be submitting the following files:

- number_guessing.py

Programming Part 1: Number Guessing

Your task is to write a program that plays a number guessing game. Download the starter code `number_guessing.py`. The computer chooses a random number between 1 and 100, then the user makes guesses until they find the right number. After every guess, the computer gives a hint by saying whether the number the user guessed was too high or too low.

For example, the interaction could look like this. The numbers after the colon are what the user/player types in.

So: the computer prints out `"Guess a number between 1 and 100: "`and then the user types in `50`.

```
>>> number_guessing()
Guess a number between 1 and 100: 50
That is too low. Try again: 75
That is too high. Try again: 67
That is too high. Try again: 58
That is too high. Try again: 54
That's right! My number was: 54
```

Some details:

- The computer should choose a number between 1 and 100 inclusive (i.e. [1, 100] to use math notation). That is, 1 is the smallest number the computer picks, and 100 is the largest number the computer picks. You may have to choose different start and stop points depending on which function you use from the random module.
- You should define a function called `number_guessing`. When this function is called, one whole round of the number guessing game is played. That is, when this function is called, the computer chooses a random number and then asks the user for a guess. If the guess is too high, the computer says so and lets the user input another guess; similarly if the guess is too low. This is repeated until the user guesses the right number.

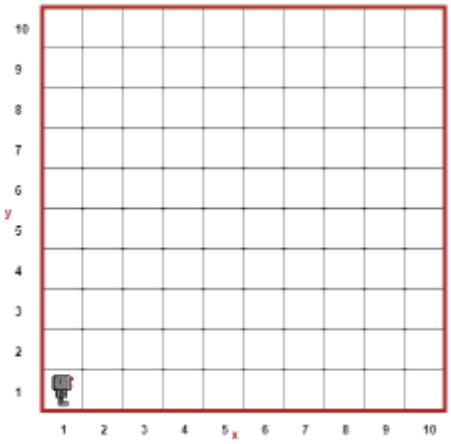
Written Questions

The code below should print "One" if the variable a equals 1, "Two" if a equals 2, "Three" if a equals 3, and "Other" if a is something else. But it prints "One" and "Other" if a equals 1, "Two" and "Other" if a equals 2, "Three" if a equals 3, and "Other" if a is something else. Explain what is wrong and fit it.

```
if a == 1:
    print("One")
if a == 2:
    print("Two")
if a == 3:
    print("Three")
else:
    print("Other")
```

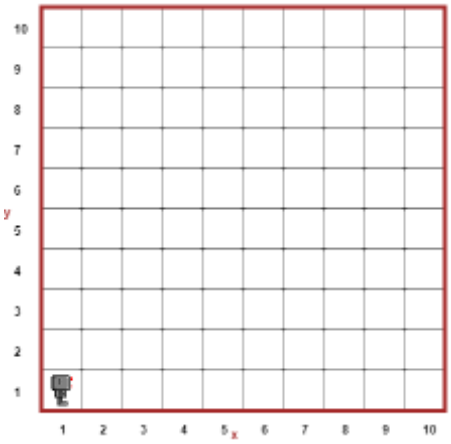
Consider the following code snippets. For each of them, what is Reeborg going to do when this code is run? You can assume that Reeborg is carrying an unlimited supply of tokens and is starting in the bottom left corner facing East as usual. Sketch the behavior of Reeborg for each of the two code snippets, and explain why Reeborg is behaving this way.

```
while front_is_clear():
    put()
    move()
```



(a) a

```
while front_is_clear():
    put()
move()
```



(b) b

.....

.....

.....

.....

.....

.....

.....

Weekly Notes

Week 1

Notes from Week 1

- Day 1 (Mon): Seymour Pappert Turtle Documentary
  - Turtle drawing activity (0.25 pts)
- Day 2 (Wed): Instructions for First Program Activity (.25 pts)
- Day 3 — Lab (Thurs): Reeborg activity: newspaper (.25 pts)

Python Examples

- in-class examples from Friday: https://drive.google.com/file/d/1UW3fMha6Ki4dDR_sidpeiV-PDeBSFEPi/view?usp=sharing
  - examples of comments, importing turtle, variables and for loops
  - also includes link to turtle library documentation

Week 2

In-class activities for week 2

- Mon: Runestone Ch. 1 review (.25 pts)
- Wed: More Turtle Drawing (.25 pts)
- Thurs (lab): Variables and Loops (.25 pts)
- Fri: Tracing & Memory Diagrams (.25 pts)

Key Resources this week

- all also found in Resources page
- Python Documentation
- Python Tutor

Examples from class

- Coding constructs: examples from class Monday - re-written a bit for clarity
- Variables and types: examples from class Thurs (lab) - in class we used PythonTutor but this is the code copied into a file in Idle

Week 3

In-class activities for week 3

- Mon: Practice with functions
  - starter code: stars0.json
  - starter code: stars1.json
- Wed: More Practice with functions & input (.25 pts)
- Thurs (lab): Functions lab (.25 pts)
  - starter code: house.py
- Fri: Study Questions(.25 pts)
  - starter code: road repair folder

Week 4

In-class activities for week 4

- Mon: Multi-branch Conditionals
  - starter code: Walk Through Field Folder
  - starter code: Harvest Folder
- Wed: example of function with optional param
  - download the above file and run in in IDLE (as a file, NOT in the shell)

```
################################################################################
# Georgia Doing (doingg@union.edu)
#
# Turtle triangle with optional param
# In class demo
################################################################################

import turtle

# example function with optional param
def turt_triangle(color = "black"):
    turtle.pencolor(color)
    for side in range(3):
        turtle.forward(75)
        turtle.left(120)
```

```
# set pensize so easier to see
turtle.pensize(5)

# draw a black triangle (default color)
turt_triangle()

# move so shapes don't overlap
turtle.teleport(100,100)

# draw a red triangle
turt_triangle("red")
```

- Thurs (lab): Practical Exam #1
  - 1 hr
  - In IDLE
  - Coding and some written answers
- Fri:

Week 4

In-class activities for week 4

- Mon: Multi-branch Conditionals
  - starter code: Walk Through Field Folder
  - starter code: Harvest Folder
- Wed: example of function with optional param
  - download the above file and run in in IDLE (as a file, NOT in the shell)

```
################################################################################
# Georgia Doing (doingg@union.edu)
#
# Turtle triangle with optional param
# In class demo
################################################################################

import turtle

# example function with optional param
def turt_triangle(color = "black"):
    turtle.pencolor(color)
    for side in range(3):
        turtle.forward(75)
        turtle.left(120)
```

```
# set pensize so easier to see
turtle.pensize(5)

# draw a black triangle (default color)
turt_triangle()

# move so shapes don't overlap
turtle.teleport(100,100)

# draw a red triangle
turt_triangle("red")
```

- Thurs (lab): Practical Exam #1
  - 1 hr
  - In IDLE
  - Coding and some written answers
- Fri:

Week 5

In-class activities for week 5

- Mon: While Loops
- Wed: Tracing while loops
- Thurs: Developers as Decision Makers
- Fri:
  - ASCII, Pig Latin
  - also due Last week's practice Problems

About

References